A Fault-Tolerant Real-Time Network Intrusion Detection Pipeline Using Apache Kafka, DataFusion, and Cassandra

Beril Aydın, Kaan Baykal
Department of Computer Engineering
TOBB University Of Economy and Technology, Ankara, Turkey

***Abstract*—**This paper presents the design, implementation, and evaluation of a locally simulated, real-time network intrusion detection pipeline built on Apache Kafka, Apache DataFusion, and Apache Cassandra, orchestrated with Docker Compose. The pipeline replays the CICIDS2017 dataset as a live stream, injecting a real timestamp per record to enable event-time tumbling-window aggregation, and applies predicate-based filtering to flag malicious flows. To satisfy the scalability, connectivity, and fault-tolerance requirements of a distributed systems design, the architecture was extended from an initial single-broker, single-node prototype to a three-broker Kafka cluster (replication factor 3) and a two-node Cassandra cluster (replication factor 2). Two independent fault-injection experiments demonstrate that the pipeline tolerates the loss of a single Kafka broker or Cassandra node during active ingestion without data loss or manual intervention. Throughput benchmarking across three send-rate tiers (100, 1,000, and 5,000 messages per second) identifies a sustainable-throughput ceiling between 1,000 and 5,000 messages per second, beyond which the pipeline exhibits transient, self-recovering backlog rather than permanent degradation. Finally, we show that DataFusion's query engine parallelizes execution across all available CPU cores via hash-partitioned aggregation, confirmed by both physical query plans and live per-core utilization measurements.

***Index Terms*—**Apache Kafka, Apache DataFusion, Apache Cassandra, stream processing, fault tolerance, distributed systems, network intrusion detection.

I. INTRODUCTION

The growing volume and velocity of network traffic has made real-time intrusion detection a data engineering problem as much as a machine-learning one: a detection rule is only useful if the infrastructure delivering it can keep pace with live traffic without losing data when a component fails. This work addresses that infrastructure problem directly, treating detection accuracy as secondary to the reliability, scalability, and fault tolerance of the pipeline that carries and processes the traffic.

We design and evaluate a containerized streaming pipeline that replays the CICIDS2017 intrusion detection dataset [1] as a live stream through Apache Kafka, processes it with Apache DataFusion, and persists results in Apache Cassandra. Consistent with a systems-engineering focus on infrastructure orchestration rather than detection-algorithm design, the dataset's ground-truth label is used as the filtering predicate; this exercises the same predicate-pushdown and partition-routing pathway that a production threshold or model-based alert would use, without requiring a bespoke detection model.

The contributions of this paper are as follows:

- A working, containerized real-time streaming pipeline integrating Apache Kafka, Apache DataFusion, and Apache Cassandra, with event-time tumbling-window aggregation.
- A fault-tolerant, multi-broker/multi-node topology, validated through controlled fault-injection experiments rather than assumed from configuration alone.
- An empirical characterization of the pipeline's sustainable throughput ceiling under progressively increasing load.
- A demonstration, via physical query plans and live CPU measurements, that DataFusion's executor model genuinely parallelizes query execution across available CPU cores.

The full implementation, including all Docker Compose configurations, producer/consumer scripts, and the Cassandra schema, is publicly available [6].

II. RELATED WORK

The CICIDS2017 dataset, introduced by Sharafaldin, Lashkari, and Ghorbani [1] as a contemporary, realistic benchmark for intrusion detection research, has been widely adopted in subsequent studies, which frequently focus on the accuracy of machine learning models rather than the underlying data engineering infrastructure required to process millions of records in real time. Kumar et al. [2] address this infrastructure gap directly, proposing a distributed intrusion detection system built on Apache Kafka and Apache Spark, using Kafka's streaming platform for reliable, low-latency data delivery combined with Spark MLlib for parallelized classification. An earlier empirical study by Tun, Nyaung, and Phyu [3] similarly benchmarks the performance of Kafka-Spark Streaming pipelines for intrusion-detection transaction processing. Both works, however, rely on JVM-heavy frameworks such as Apache Spark Structured Streaming; recent advances in Rust-based query engines, notably Apache DataFusion [4], have demonstrated significant improvements in memory efficiency and execution speed relative to JVM-based engines through an Arrow-native, vectorized execution model. Literature integrating DataFusion with Apache Kafka and NoSQL databases such as Cassandra for continuous cybersecurity monitoring remains limited. This paper aims to bridge that gap by demonstrating a micro-batching integration strategy for these three technologies, and by empirically validating the resulting

system's fault tolerance and throughput characteristics, which prior work in this space does not report.

III. SYSTEM DESIGN AND IMPLEMENTATION

A. Dataset and Preprocessing

The CICIDS2017 dataset [1] was obtained from Kaggle [5] as eight static CSV files, totaling 2,830,743 rows across 79 columns (78 network traffic features plus the class label), approximately 880 MB. Exploratory analysis confirmed the dataset's structure and quality: missing or infinite values were detected in only 2,867 rows (0.1013%), concentrated in the Flow Bytes/s and Flow Packets/s columns; the Destination Port feature is heavily skewed (33.84% Port 53, 21.86% Port 80, 17.86% Port 443), a realistic imbalance deliberately chosen as the Kafka partitioning key so that partition-level load distribution and hot-partition behavior could be stress-tested, rather than being masked by an artificially uniform key; and the label distribution is imbalanced, with 80.30% of records labeled BENIGN and the remainder distributed across attack types such as DoS Hulk (8.16%), PortScan (5.61%), and DDoS (4.52%). A preprocessing stage corrects duplicate column names, repairs character-encoding corruption in attack labels, and removes rows containing missing or infinite values in any column, rather than imputing them.

B. Pipeline Architecture

The pipeline orchestrates four operational layers. A Python producer replays cleaned CSV records to Kafka, injecting a real UTC timestamp into each row at send time to simulate a live stream from data that otherwise carries no timestamp information. Apache Kafka buffers and partitions the stream by Destination Port. A DataFusion-based consumer accumulates Kafka messages into micro-batches, applies rule-based filtering to isolate non-benign flows, and computes event-time tumbling-window aggregations by truncating each batch's injected timestamps to a fixed granularity, rather than using the wall-clock time at which the batch happens to be processed. Apache Cassandra persists both the flagged anomalies and the windowed traffic metrics.

C. From Single-Node to Multi-Node Topology

An initial prototype ran a single Kafka broker and a single Cassandra node. While sufficient to validate the data flow and windowing logic, this topology cannot demonstrate fault tolerance: the loss of the single broker or node is equivalent to the entire pipeline stopping, with no replica available for recovery. For the evaluation reported in Section IV, the architecture was extended to three Kafka brokers with topic replication factor 3 and min.insync.replicas set to 2, and two Cassandra nodes with keyspace replication factor 2. Each service's JVM heap was explicitly capped to fit within a memory-constrained local development environment.

D. Event-Time Windowing

Tumbling windows are computed by truncating each record's injected timestamp to a one-second boundary and aggregating with a standard GROUP BY, rather than with SQL window (OVER) functions, which operate per-row rather than collapsing rows into window summaries. An early implementation keyed windowed metrics only by destination port and window start; because a Cassandra INSERT is an upsert, two different micro-batches landing on the same window silently overwrote one another, discarding the earlier batch's counts. This was corrected by adding a per-batch identifier to the primary key, so each micro-batch's contribution to a window is preserved as its own row, with totals across batches computed at query time via a SUM aggregation.

IV. EVALUATION AND RESULTS

A. Fault-Tolerance Evaluation

Two independent fault-injection experiments were conducted against the multi-node topology, each replaying 3,000 records from the CICIDS2017 PortScan subset at 50 messages per second while a component was killed approximately one-third of the way through the stream.

1) Kafka Broker Failure: With the topic configured at replication factor 3 across three brokers and min.insync.replicas=2, one broker was stopped mid-stream.

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ python3 producer/produce.py \
  --file data/clean/Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv \
  --rate 50 --limit 3000
  ... 500 messages sent
  ... 1,000 messages sent
%6|1785142781.265|FAIL|rdkafka#producer-1| [thrd:localhost:9093/2]: localhost:9093/2: Disconnected: connection closed by peer: receive 0 after POLLIN (after 29471ms
 in state UP)
%6|1785142781.361|FAIL|rdkafka#producer-1| [thrd:localhost:9093/2]: localhost:9093/2: Disconnected: connection reset by peer (after 8ms in state APIVERSION_QUERY)
%6|1785142781.646|FAIL|rdkafka#producer-1| [thrd:localhost:9093/2]: localhost:9093/2: Disconnected: connection reset by peer (after 1ms in state APIVERSION_QUERY, 1
 identical error(s) suppressed)
  ... 1,500 messages sent
%3|1785142782.691|FAIL|rdkafka#producer-1| [thrd:localhost:9093/2]: localhost:9093/2: Connect to ipv4#127.0.0.1:9093 failed: Connection refused (after 0ms in state
CONNECT)
  ... 2,000 messages sent
  ... 2,500 messages sent
  ... 3,000 messages sent

Done. Total sent: 3,000 messages
berilay@BerilsHuawei:~/intrusion-detection-pipeline$
```

Fig. 1. Producer terminal during the broker fault injection. Connection errors to the stopped broker are logged, but message delivery continues uninterrupted; all 3,000 messages are sent.

```
  Batch processed: 100 msgs, 0 anomalies, 11 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 24 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 26 window/port groups written to Cassandra
^C
Done. Total batches: 30
Traceback (most recent call last):
  File "/home/berilay/intrusion-detection-pipeline/datafusion/consume_process.py", line 181, in <module>
    main()
  File "/home/berilay/intrusion-detection-pipeline/datafusion/consume_process.py", line 152, in main
    msg = consumer.poll(timeout=5.0)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt

berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it cassandra1 cqlsh -e "SELECT SUM(total_flows) as total FROM intrusion_detection.windowed_traffic
_metrics;"

 total
-------
  3000

(1 rows)

Warnings :
Aggregation query used without partition key

berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it kafka1 kafka-topics --describe --topic network-traffic --bootstrap-server localhost:9092
[2026-07-27 09:29:19,569] WARN [AdminClient clientId=adminclient-1] Connection to node 3 (localhost/127.0.0.1:9094) could not be established. Broker may not be avai
lable. (org.apache.kafka.clients.NetworkClient)
Topic: network-traffic  TopicId: gSzWpU0jQwu4ffCa9zqLtw PartitionCount: 3       ReplicationFactor: 3    Configs: min.insync.replicas=2
        Topic: network-traffic  Partition: 0    Leader: 1       Replicas: 2,1,3 Isr: 1,3
        Topic: network-traffic  Partition: 1    Leader: 3       Replicas: 3,2,1 Isr: 3,1
        Topic: network-traffic  Partition: 2    Leader: 1       Replicas: 1,3,2 Isr: 1,3
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker start kafka2
sleep 15
docker exec -it kafka1 kafka-topics --describe --topic network-traffic --bootstrap-server localhost:9092
kafka2
[2026-07-27 09:29:49,821] WARN [AdminClient clientId=adminclient-1] Connection to node 2 (localhost/127.0.0.1:9093) could not be established. Broker may not be avai
lable. (org.apache.kafka.clients.NetworkClient)
[2026-07-27 09:29:49,826] WARN [AdminClient clientId=adminclient-1] Connection to node 3 (localhost/127.0.0.1:9094) could not be established. Broker may not be avai
lable. (org.apache.kafka.clients.NetworkClient)
Topic: network-traffic  TopicId: gSzWpU0jQwu4ffCa9zqLtw PartitionCount: 3       ReplicationFactor: 3    Configs: min.insync.replicas=2
        Topic: network-traffic  Partition: 0    Leader: 1       Replicas: 2,1,3 Isr: 1,3,2
        Topic: network-traffic  Partition: 1    Leader: 3       Replicas: 3,2,1 Isr: 3,1,2
        Topic: network-traffic  Partition: 2    Leader: 1       Replicas: 1,3,2 Isr: 1,3,2
```

Fig. 2. Verification after the run: the sum of total_flows in Cassandra equals 3,000 (no message loss), and the topic's in-sync replica list is fully restored within 15 seconds of restarting the broker.

Neither the producer nor the consumer terminated. Partition leadership for all three partitions was automatically reassigned away from the stopped broker. All 3,000 messages were confirmed processed via three independent counts: the producer's send total, the consumer's cumulative batch count, and the sum of total_flows recorded in Cassandra with zero loss. Upon restart, the broker rejoined the in-sync replica set within 15 seconds.

2) Cassandra Node Failure: With the keyspace configured at replication factor 2 across two nodes, an initial attempt stopped the driver's original contact point mid-stream.

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ python3 datafusion/consume_process.py --batch-size 100
Consumer started, waiting for messages...
  Batch processed: 100 msgs, 0 anomalies, 26 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 38 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 30 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 24 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 11 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 22 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 9 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 6 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 8 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 5 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 9 window/port groups written to Cassandra

Done. Total batches: 11
Traceback (most recent call last):
  File "/home/berilay/intrusion-detection-pipeline/datafusion/consume_process.py", line 181, in <module>
    main()
  File "/home/berilay/intrusion-detection-pipeline/datafusion/consume_process.py", line 169, in main
    process_batch(buffer, ctx, cassandra_session)
  File "/home/berilay/intrusion-detection-pipeline/datafusion/consume_process.py", line 111, in process_batch
    cassandra_session.execute("""
  File "cassandra/cluster.py", line 2678, in cassandra.cluster.Session.execute
  File "cassandra/cluster.py", line 5049, in cassandra.cluster.ResponseFuture.result
cassandra.cluster.NoHostAvailable: ('Unable to complete the operation against any hosts', {<Host: 127.0.0.1:9042 datacenter1>: ConnectionShutdown('Connectio
n to 127.0.0.1:9042 is closed')})
```

Fig. 3. Initial attempt: the consumer crashed with NoHostAvailable when the contact-point node was stopped. The driver had been configured with a single contact point, so it had no alternative host to fail over to even though a second, in-sync replica was available.

This was corrected by publishing each Cassandra node on a distinct loopback address with a matching broadcast RPC address, and configuring the driver with both addresses as contact points, ensuring a live path to the cluster regardless of which node is originally contacted. The experiment was then repeated:

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ python3 producer/produce.py \
  --file data/clean/Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv \
  --rate 50 --limit 3000
  ... 500 messages sent
  ... 1,000 messages sent
  ... 1,500 messages sent
  ... 2,000 messages sent
  ... 2,500 messages sent
  ... 3,000 messages sent

Done. Total sent: 3,000 messages
```

Fig. 4. Producer terminal: all 3,000 messages sent without error.

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ python3 datafusion/consume_process.py --batch-size 100
Consumer started, waiting for messages...
  Batch processed: 100 msgs, 0 anomalies, 16 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 30 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 43 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 22 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 10 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 23 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 9 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 6 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 7 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 5 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 8 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 5 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 22 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 20 window/port groups written to Cassandra
  Batch processed: 100 msgs, 4 anomalies, 30 window/port groups written to Cassandra
  Batch processed: 100 msgs, 3 anomalies, 47 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 58 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 24 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 17 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 29 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 44 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 22 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 17 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 9 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 8 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 12 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 12 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 12 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 26 window/port groups written to Cassandra
  Batch processed: 100 msgs, 0 anomalies, 25 window/port groups written to Cassandra
^C
Done. Total batches: 30
```

Fig. 5. Consumer terminal: the contact-point node was stopped after roughly 1,000 messages. Unlike the first attempt, no disconnect or retry messages appear at all; batch processing continues without visible interruption. All 30 batches (3,000 messages) were processed.

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it cassandra1 nodetool status
Datacenter: datacenter1
=======================
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load        Tokens  Owns (effective)  Host ID                               Rack
UN  172.21.0.5  124.69 KiB  16      100.0%            2de234bc-c8ca-4c50-8f43-64b6b7499994  rack1
UN  172.21.0.2  161.46 KiB  16      100.0%            e7593d46-6560-4121-93bc-ee1839884783  rack1

berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it cassandra2 cqlsh -e "SELECT COUNT(*) FROM intrusion_detection.flagged_anomalies;"

 count
-------
     7

(1 rows)

Warnings :
Aggregation query used without partition key

berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it cassandra1 cqlsh -e "SELECT COUNT(*) FROM intrusion_detection.flagged_anomalies;"

 count
-------
     7

(1 rows)

Warnings :
Aggregation query used without partition key
```

Fig. 6. After restarting the stopped node, nodetool status confirms both nodes returned to the Up/Normal state, and querying flagged_anomalies independently on both nodes returns an identical count (7 rows), confirming that data written to the surviving node during the outage was correctly replicated back to the recovered node.

The consumer continued writing to the surviving node without interruption or error. All 3,000 messages were confirmed written, including 7 correctly flagged PortScan anomalies, replicated identically to both nodes after recovery.

3) Summary: Table I summarizes both experiments.

TABLE I FAULT-INJECTION EXPERIMENT SUMMARY

Metric	Kafka Broker Failure	Cassandra Node Failure
Messages sent	3,000	3,000
Messages processed	3,000 (no loss)	3,000 (no loss)
Producer/consumer stopped?	No	No
Recovery confirmed by	ISR restored in 15 s	Both nodes UN; replicated data identical (7 rows)

B. Full-Scale Performance Benchmarking

To identify the pipeline's maximum sustainable throughput, the producer's send rate was progressively increased (100 → 1,000 → 5,000 msgs/sec) against increasing volumes of the PortScan dataset. At each tier, Kafka consumer group lag was measured both mid-stream, while the producer was still sending, and after producer completion, to distinguish transient burst lag from a genuine, unrecoverable bottleneck.

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it kafka1 kafka-consumer-groups --bootstrap-server kafka1:29092   --describe --group dataf
usion-processor

GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                     HOST
 CLIENT-ID
datafusion-processor network-traffic 0          2118            2468            350             rdkafka-f483cce7-e9a9-4e0f-a526-28bcde929f1c /172.21.0.1
 rdkafka
datafusion-processor network-traffic 1          199             206             7               rdkafka-f483cce7-e9a9-4e0f-a526-28bcde929f1c /172.21.0.1
 rdkafka
datafusion-processor network-traffic 2          183             189             6               rdkafka-f483cce7-e9a9-4e0f-a526-28bcde929f1c /172.21.0.1
 rdkafka
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it kafka1 kafka-consumer-groups --bootstrap-server kafka1:29092   --describe --group dataf
usion-processor

GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                     HOST
 CLIENT-ID
datafusion-processor network-traffic 0          4233            4233            0               rdkafka-f483cce7-e9a9-4e0f-a526-28bcde929f1c /172.21.0.1
 rdkafka
datafusion-processor network-traffic 1          384             384             0               rdkafka-f483cce7-e9a9-4e0f-a526-28bcde929f1c /172.21.0.1
 rdkafka
datafusion-processor network-traffic 2          383             383             0               rdkafka-f483cce7-e9a9-4e0f-a526-28bcde929f1c /172.21.0.1
 rdkafka
```

Fig. 7. Tier 1 (100 msgs/sec, 5,000 messages): mid-stream lag (top) totals 363 messages; final lag (bottom) is 0.

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it kafka1 kafka-consumer-groups --bootstrap-server kafka1:29092   --describe --group dataf
usion-processor

GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                     HOST
 CLIENT-ID
datafusion-processor network-traffic 0          37381           38271           890             rdkafka-4e5fed44-ec4b-4f82-8dbd-cc034fd3faa6 /172.21.0.1
 rdkafka
datafusion-processor network-traffic 1          3297            3348            51              rdkafka-4e5fed44-ec4b-4f82-8dbd-cc034fd3faa6 /172.21.0.1
 rdkafka
datafusion-processor network-traffic 2          3322            3381            59              rdkafka-4e5fed44-ec4b-4f82-8dbd-cc034fd3faa6 /172.21.0.1
 rdkafka
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it kafka1 kafka-consumer-groups --bootstrap-server kafka1:29092   --describe --group dataf
usion-processor

GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                     HOST
 CLIENT-ID
datafusion-processor network-traffic 0          38271           38271           0               rdkafka-4e5fed44-ec4b-4f82-8dbd-cc034fd3faa6 /172.21.0.1
 rdkafka
datafusion-processor network-traffic 1          3348            3348            0               rdkafka-4e5fed44-ec4b-4f82-8dbd-cc034fd3faa6 /172.21.0.1
 rdkafka
datafusion-processor network-traffic 2          3381            3381            0               rdkafka-4e5fed44-ec4b-4f82-8dbd-cc034fd3faa6 /172.21.0.1
 rdkafka
```

Fig. 8. Tier 2 (1,000 msgs/sec, 20,000 messages): mid-stream lag (top) totals 1,000 messages; final lag (bottom) is 0.

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it kafka1 kafka-consumer-groups --bootstrap-server kafka1:29092 \
  --describe --group datafusion-processor

GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                     HOST
 CLIENT-ID
datafusion-processor network-traffic 0          30604           64358           33754           rdkafka-49140e0c-2c1a-415a-84d2-d15f1c5f32fa /172.21.0.1
 rdkafka
datafusion-processor network-traffic 1          2939            5245            2306            rdkafka-49140e0c-2c1a-415a-84d2-d15f1c5f32fa /172.21.0.1
 rdkafka
datafusion-processor network-traffic 2          2957            5397            2440            rdkafka-49140e0c-2c1a-415a-84d2-d15f1c5f32fa /172.21.0.1
 rdkafka
```

Fig. 9. Tier 3 (5,000 msgs/sec, 50,000 messages), mid-stream: lag has grown to approximately 38,500 messages.

```
berilay@BerilsHuawei:~/intrusion-detection-pipeline$ docker exec -it kafka1 kafka-consumer-groups --bootstrap-server kafka1:29092 \
  --describe --group datafusion-processor

GROUP                TOPIC           PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG             CONSUMER-ID                                     HOST
 CLIENT-ID
datafusion-processor network-traffic 0          64358           64358           0               rdkafka-49140e0c-2c1a-415a-84d2-d15f1c5f32fa /172.21.0.1
 rdkafka
datafusion-processor network-traffic 1          5245            5245            0               rdkafka-49140e0c-2c1a-415a-84d2-d15f1c5f32fa /172.21.0.1
 rdkafka
datafusion-processor network-traffic 2          5397            5397            0               rdkafka-49140e0c-2c1a-415a-84d2-d15f1c5f32fa /172.21.0.1
 rdkafka
```

Fig. 10. Tier 3, after producer completion: lag on all three partitions fully resolves to 0 within roughly 35 seconds.

TABLE II THROUGHPUT BENCHMARK SUMMARY

Tier	Rate	Total Msgs	Mid-Stream Lag	Final Lag
1	100/s	5,000	363 (~7%)	0
2	1,000/s	20,000	1,000 (~5%)	0
3	5,000/s	50,000	~38,500 (~77%)	0

Percentages are calculated using each tier's own --limit value (5,000 / 20,000 / 50,000), rather than the cumulative Kafka log-end-offset. Since the topic was not reset between tiers, the cumulative offset also includes traffic from previous tiers, which would make later-tier lag percentages appear smaller than they actually are. At 100 and 1,000 msgs/sec, lag remained small relative to the tier volume (363/5,000 ≈ 7% and 1,000/20,000 = 5%) and fully recovered to zero shortly after the producer finished. At 5,000 msgs/sec, mid-stream lag reached 38,500/50,000 = 77%, more than an order of magnitude higher than at the previous tiers. However, only three rates were tested, so intermediate rates are needed to determine the exact throughput threshold. The results place the sustainable-throughput ceiling between 1,000 and 5,000 msgs/sec. Importantly, lag returned to zero at all three tiers after ingestion stopped, showing that the pipeline can recover from temporary backlog rather than accumulating an unbounded backlog.

C. CPU Core Utilization and Parallelism

DataFusion parallelizes query execution via the target_partitions setting, which defaults to the number of available CPU cores; the physical plan splits a query into that many independent execution partitions, each processed concurrently by DataFusion's Rust-native thread pool. On an 8-core machine, running the same port-level aggregation query used by the pipeline's consumer against a full day's cleaned CSV file produced the following physical plan:

```
AggregateExec: mode=FinalPartitioned, gby=[Destination Port]
  RepartitionExec: partitioning=Hash([Destination Port], 8),
                   input_partitions=8
    AggregateExec: mode=Partial, gby=[Destination Port]
      DataSourceExec: file_groups={8 groups: [0..10782677],
                      [10782677..21565354], ...}
```

The source file is split into 8 byte-range groups and processed through a two-stage, hash-partitioned aggregation (Partial -> Repartition -> FinalPartitioned), confirming that DataFusion genuinely distributes the scan and aggregation work across 8 independent execution partitions rather than executing the query on a single thread. To confirm this translates into real, concurrent CPU usage rather than merely a configured partition count, per-core utilization was sampled while the query executed:

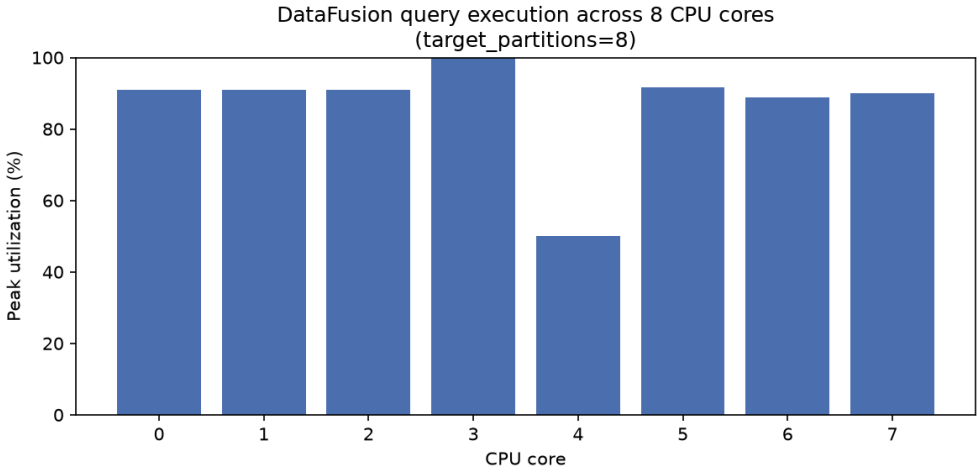

Fig. 11. Peak per-core CPU utilization sampled while DataFusion executed the grouped aggregation query with target_partitions=8. All 8 cores were active simultaneously.

TABLE III PEAK PER-CORE CPU UTILIZATION (SINGLE-RUN SAMPLE)

Core	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Peak %	90.9	90.9	90.9	100.0	50.0	91.7	88.9	90.0

All 8 logical cores exceeded 25% utilization during the query, with 6 of the 8 reaching 88–100%. Together with the physical plan's explicit RepartitionExec(8) stage, this confirms that DataFusion's executor model maps query partitions onto separate OS threads that the scheduler distributes across distinct physical cores, rather than executing the aggregation sequentially.

Reproducibility note: The specific peak-utilization values and chart shown in Fig. 11 and the accompanying table are from a single execution. Because thread-to-core assignment is determined by the operating system's scheduler and is not pinned by DataFusion, re-running the identical query produces a different distribution of peak utilization across the 8 cores each time, the table and chart in this paper will not reproduce exactly if regenerated. What is consistent across runs is that all (or nearly all) cores exceed the 25% activity threshold, confirming genuine multi-core parallelism rather than a fixed, cherry-picked result. This also makes the measurement well suited to a live demonstration: the accompanying script can be re-executed at presentation time to produce a fresh, unscripted measurement rather than a static screenshot.

V. CONCLUSION AND FUTURE WORK

This paper presented the design, implementation, and empirical evaluation of a real-time network intrusion detection pipeline built on Apache Kafka, Apache DataFusion, and Apache Cassandra. The architecture was extended from a single-broker, single-node prototype to a fault-tolerant, multi-node topology, and two independent fault-injection experiments confirmed that the pipeline tolerates the loss of a single Kafka broker or Cassandra node during active ingestion without data loss or manual intervention. Throughput benchmarking across three load tiers identified a sustainable-throughput ceiling between 1,000 and 5,000 messages per second, with the pipeline exhibiting graceful, self-recovering degradation rather than unbounded backlog beyond that point. Finally, we demonstrated, through both physical query plans and live CPU measurements, that DataFusion's executor model genuinely parallelizes query execution across all available CPU cores.

Several directions remain for future work. First, the anomaly-filtering predicate currently relies on the dataset's ground-truth label; a natural extension is to replace this with a genuine unsupervised or statistical detection rule that does not depend on pre-existing labels, more closely reflecting a production deployment. Second, the throughput ceiling identified in Section IV-B could be narrowed with additional intermediate load tiers. Third, migrating message serialization from JSON to a binary format such as Apache Arrow or Avro may reduce deserialization overhead and further raise the sustainable throughput ceiling. Finally, a live demonstration combining the fault-injection and CPU-parallelism experiments reported here, run end-to-end against the full 2.8-million-record dataset, is planned as the concluding validation of this work.

REFERENCES

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in Proc. 4th Int. Conf. Information

Systems Security and Privacy (ICISSP), Portugal, 2018, pp. 108–116.

[2] K. M. K. Kumar, M. Sreekar, K. Ullas, and S. Muthuraman, “Distributed Intrusion Detection System using Kafka and Spark Streaming,” in Proc. 2025 Int. Conf. Visual Analytics and Data Visualization (ICVADV), 2025.

[3] M. T. Tun, D. E. Nyaung, and M. P. Phyu, “Performance Evaluation of Intrusion Detection Streaming Transactions Using Apache Kafka and Spark Streaming,” in Proc. 2019 Int. Conf. Advanced Information Technologies (ICAIT), 2019, pp. 25–30, doi: 10.1109/AITC.2019.8920960.

[4] A. Lamb et al., “Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine,” in Companion of the 2024 Int. Conf. on Management of Data (SIGMOD Companion ’24), ACM, 2024, doi: 10.1145/3626246.3653368.

[5] Kaggle, “Network Intrusion Dataset (CICIDS2017),” [Online]. Available: https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset. [Accessed: 2026].

[6] B. Aydın and K. Baykal, “Real-Time Network Intrusion Detection Pipeline,” GitHub repository. [Online]. Available: https://github.com/Berilay6/Real-Time-Network-Intrusion-Detection-Pipeline.